TaskForge — Smart Task Management System

Detailed Project Report

Built with Next.js · TypeScript · MongoDB · SOLID Principles

April 2026

1. Introduction

TaskForge is a modern, full-stack task management system designed for team collaboration. The application enables administrators to create projects, manage team members, assign tasks with priorities and deadlines, and monitor progress through a comprehensive dashboard. Team members receive notifications, accept project invitations, and update their task statuses through a dedicated member interface. The system is built with a strong emphasis on software engineering principles, implementing SOLID design principles, the Service-Repository architectural pattern, and secure JWT-based authentication.

2. Project Objectives

- Build a scalable task management platform with role-based access control
- Implement secure authentication using JWT tokens and bcrypt password hashing
- Follow SOLID principles and clean architecture for maintainability
- Create an intuitive admin dashboard for project and team management
- Provide a member interface for task tracking and notifications
- Enforce business rules such as unique project names and task title constraints
- Implement an approval workflow for new member registrations

3. Tech Stack

Layer	Technology	Purpose
Frontend Framework	Next.js 16 (App Router)	SSR, file-based routing, API routes
UI Library	React 19	Component-based UI with hooks and context
Language	TypeScript 5 (strict)	Static typing, interfaces, generics
Styling	Tailwind CSS 4	Utility-first CSS with dark mode
Animation	Framer Motion 12	Page transitions, scroll animations
Icons	Lucide React	SVG icon library
Database	MongoDB + Mongoose 9	NoSQL document database with ODM
Auth (Server)	jsonwebtoken 9	JWT token generation and verification
Auth (Edge)	jose 6	Lightweight JWT verification for middleware
Password Security	bcryptjs 3	Salted password hashing (12 rounds)
Date Handling	date-fns 4	Date formatting, deadline calculations

4. System Architecture

TaskForge employs a **Layered Architecture** with the **Service-Repository Pattern**:

```
┌──────────────────────────────────────────┐
│ Presentation Layer │
│ React Components, Pages, Layouts │
├──────────────────────────────────────────┤
│ API Layer │
│ Next.js API Route Handlers │
├──────────────────────────────────────────┤
│ Service Layer │
│ AuthService, TaskService, UserService, │
│ ProjectService, InvitationService, │
│ NotificationService │
├──────────────────────────────────────────┤
│ Repository Layer │
│ UserRepo, TaskRepo, ProjectRepo, │
│ InvitationRepo, NotificationRepo │
├──────────────────────────────────────────┤
│ Data Layer │
│ MongoDB via Mongoose Schemas │
└──────────────────────────────────────────┘
```

- **Presentation:** React components, Context providers (Auth, Theme), Framer Motion animations
- **API:** Route handlers validate input, delegate to services, return JSON responses
- **Service:** Business logic, validation, uniqueness enforcement, cross-repository orchestration
- **Repository:** Database CRUD operations, domain-specific queries, Mongoose model access
- **Data:** Mongoose schemas with constraints, indexes, and automatic timestamps

5. SOLID Principles

S — Single Responsibility Principle

Class	Single Responsibility
BaseEntity	ID generation and timestamp management only
BaseRepository	Generic CRUD database operations only
BaseService	Error handling, logging, validation only
AuthService	Authentication: login, signup, JWT only
UserService	User CRUD, approval, password hashing only
TaskService	Task creation, assignment, validation only
ProjectService	Project CRUD, member management only
InvitationService	Invitation workflow only
NotificationService	Notification management only

O — Open/Closed Principle

- `BaseEntity` provides abstract `validate()`. Each entity extends with domain-specific rules without modifying the base.
- `BaseRepository` provides CRUD. TaskRepository adds `findByAdminAndTitle()`, `findByAssignee()` without altering the base.
- `BaseService` provides `execute()`. Concrete services extend with domain logic without changing the base.

L — Liskov Substitution Principle

- All entities extend BaseEntity and implement IEntity — any can substitute where IEntity is expected.
- All repositories extend BaseRepository and implement IRepository — any can substitute where IRepository is expected.
- All services extend BaseService with consistent execute() and validateRequired() patterns.

I — Interface Segregation Principle

- `IReadable<T>` — findById(), findAll()
- `IWritable<T>` — create(), update()
- `IDeletable` — delete()
- `IRepository<T>` — composes all three + findByFilter()

> A read-only module can depend on just `IReadable<ITask>` — consumers only depend on what they need.

D — Dependency Inversion Principle

- API routes depend on service interfaces (IAuthService, ITaskService, IUserService) via getInstance().
- Services depend on repository abstractions, not directly on MongoDB/Mongoose.
- Edge middleware uses `jose` for JWT, decoupled from `jsonwebtoken` in services.

> **Fix Applied:** Users API routes previously called UserRepository directly (DIP violation). A dedicated UserService was created to restore proper layering.

6. Design Patterns

Pattern	Implementation
Repository Pattern	All repositories abstract MongoDB behind IRepository<T>
Service Pattern	Business logic encapsulated in dedicated service classes
Singleton Pattern	Private constructors + getInstance() for all repos and services
Template Method	BaseEntity.validate() contract; subclasses implement rules
Strategy Pattern	Navbar selects link sets based on user role
Observer Pattern	Task API triggers NotificationService on assignment
Provider Pattern	React Context for Auth and Theme global state
Factory Method	BaseEntity.generateId() encapsulates ID creation

7. Database Design

Collections and Constraints

Collection	Key Fields	Constraints
Users	name, email, password, role, isApproved	email: unique, lowercase
Tasks	title, description, status, priority, deadline, assignedTo, assignedBy, projectId	Compound index: title + assignedBy (unique)
Projects	name, description, ownerId, memberIds	name: globally unique
Invitations	projectId, adminId, memberId, status	App-level duplicate check
Notifications	userId, type, message, read, relatedTaskId	type enum: task_assigned/task_completed

Entity Relationships

Relationship	Type	Description
User → Project	One-to-Many	A user owns multiple projects (via ownerId)
User ↔ Project	Many-to-Many	Users are members of projects (via memberIds)
Project → Task	One-to-Many	A project contains multiple tasks
User → Task	One-to-Many	Tasks assigned to and assigned by users
User → Notification	One-to-Many	Users receive notifications
User → Invitation	One-to-Many	Users send/receive invitations

8. Authentication & Security

Authentication Flow

```
User submits login → POST /api/auth/login
→ AuthService.login()
→ UserRepository.findByEmail() → MongoDB
→ Check isApproved (members must be approved)
→ bcrypt.compare() password verification
→ jwt.sign() generates JWT (7-day expiry)
→ Set HTTP-only cookie (taskforge-token)
→ AuthProvider stores in context + localStorage
```

```
→ Edge Middleware validates JWT on subsequent requests
→ Role-based redirect: admin→/dashboard, member→/member
```

Security Measures

- **Password Hashing:** bcryptjs with 12 salt rounds
- **HTTP-Only Cookies:** Prevents XSS on JWT tokens
- **Edge Middleware:** JWT verification on every protected route using jose
- **Role-Based Routing:** Admin→/dashboard, Member→/member with auto-redirect
- **Member Approval:** New signups require admin approval
- **Password Stripping:** Mongoose toJSON transform removes passwords from responses

9. API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	Authenticate user, return JWT
POST	/api/auth/signup	Register new user
POST	/api/auth/logout	Clear auth cookie
GET	/api/tasks	Get tasks (?assignedTo, ?projectId)
POST	/api/tasks	Create task + auto-notification
PATCH	/api/tasks/:id	Update task/status
DELETE	/api/tasks/:id	Delete task
GET	/api/projects	Get all projects
POST	/api/projects	Create project (unique name)
GET	/api/users	Get users (no passwords)
POST	/api/users	Create user
PUT	/api/users/:id	Update/approve user
DELETE	/api/users/:id	Reject/delete user
GET	/api/invitations	Get invitations (?memberId)
POST	/api/invitations	Create invitation
PATCH	/api/invitations/:id	Accept/decline
GET	/api/notifications	Get notifications (?userId)
PATCH	/api/notifications	Mark all read (?userId)

10. Application Workflows

Task Creation Flow

```
Admin fills task form → POST /api/tasks
→ TaskService.createTask()
→ validateRequired(['title', 'description'])
→ Check uniqueness per admin (findByAdminAndTitle)
→ Check uniqueness per project (findByProjectAndTitle)
→ TaskRepository.create() → MongoDB insert
→ If assignedTo:
→ NotificationService.createNotification()
→ 201 Response → Dashboard re-fetches → UI updates
```

Member Approval Flow

```
New member signs up → isApproved = false
→ Login attempt rejected ("pending approval")
→ Admin sees pending members on dashboard
→ Approve: UserService.approveUser() → isApproved = true
→ Reject: UserService.rejectUser() → user deleted
```

Invitation Flow

```
Admin invites member → POST /api/invitations
→ Check for duplicate pending invitation
→ InvitationRepository.create()
→ Member sees invitation
→ Accept → updateStatus('accepted')
→ ProjectService.addMember() → $addToSet
→ Decline → updateStatus('declined')
```

11. Frontend Architecture

Component Hierarchy

```
RootLayout
├── ThemeProvider — dark/light mode (React Context)
├── AuthProvider — authentication state (React Context)
├── BackgroundShapes — animated decorative elements
├── Navbar — glassmorphism, role-based, responsive
└── Pages
```

```
├── Home (/) — Hero, Stats, Features, Testimonials
├── Login (/login) — Auth form
├── Signup (/signup) — Registration + role selection
├── Dashboard (/dashboard) — Admin management
└── Member (/member) — Tasks, notifications, invitations
```

Reusable Components

Component	Purpose
AnimatedCard	Scroll-triggered fade-in animation wrapper
ParallaxSection	Parallax scroll effect wrapper
FormInput	Reusable form input with icon support
StatsCard	Animated counter statistics card
TaskCard	Task with status/priority/deadline indicators
TeamMemberCard	Member card with approve/reject actions

12. OOP Concepts

Concept	Implementation
Abstraction	BaseEntity, BaseRepository, BaseService hide details behind abstract methods and interfaces
Encapsulation	Entity classes bundle data + validation. Singletons use private constructors
Inheritance	All entities→BaseEntity, repos→BaseRepository, services→BaseService
Polymorphism	Each entity overrides validate() with domain-specific rules
Generics	BaseRepository<T>, IRepository<T>, IReadable<T> for type-safe reuse

13. SDLC Methodology

The project follows **Agile** methodology:

1. **Planning** — Core requirements: auth, task management, roles
2. **Analysis** — User needs, entity relationships, specifications
3. **Design** — Architecture, class, ER, sequence, data flow diagrams
4. **Implementation** — Next.js + TypeScript with SOLID principles
5. **Testing** — Jest and React Testing Library
6. **Deployment** — Vercel + MongoDB Atlas
7. **Maintenance** — Iterative improvements and feature additions

14. Conclusion

TaskForge demonstrates a professionally architected full-stack web application that strictly adheres to SOLID principles, established design patterns, and clean architecture practices. The Service-Repository pattern ensures clear separation of concerns. TypeScript interfaces provide strong contracts between layers. The system is modular and extensible — new entities, services, or repositories can be added without modifying existing code, following the Open/Closed Principle. The combination of JWT authentication, role-based edge middleware, bcrypt password hashing, and HTTP-only cookies provides a robust, production-grade security foundation.